

JavaLibrary: Sistema de Gerenciamento de Biblioteca

Ana Clara Stolses

*Instituto de Ciências Matemáticas e de Computação
Universidade de São Paulo
São Paulo, Brasil
NUSP: 15654835*

Isabela Lima

*Instituto de Ciências Matemáticas e de Computação
Universidade de São Paulo
São Paulo, Brasil
NUSP: 15678780*

Abstract—Este relatório descreve o desenvolvimento do **JavaLibrary**, um sistema de gestão de biblioteca implementado em Java com interface gráfica Swing. O sistema permite o cadastro de livros e usuários, o controle de empréstimos e devoluções, o cálculo automático de multas por atraso e a geração de relatórios gerenciais, além de oferecer controle de acesso por perfil de usuário. O documento apresenta os requisitos do projeto, a arquitetura adotada, as decisões de implementação, os mockups das principais telas, o plano e os resultados dos testes manuais realizados, os procedimentos de build e as limitações identificadas, encerrando com sugestões de melhorias para versões futuras.

Index Terms—Java, Swing, biblioteca, persistência, programação orientada a objetos, sistema de gestão.

I. INTRODUÇÃO

A gestão manual de acervos de biblioteca está sujeita a erros de controle de estoque, atraso na cobrança de multas e dificuldade de geração de relatórios consolidados. O **JavaLibrary** foi concebido para automatizar esse processo, oferecendo controle de acervo, cadastro de usuários, registro de empréstimos e devoluções, cálculo automático de multas e geração de relatórios de forma integrada.

O projeto foi desenvolvido como atividade acadêmica da disciplina de Programação Orientada a Objetos (POO), com o objetivo de aplicar, em um sistema funcional, conceitos como encapsulamento, herança, polimorfismo e abstração, além de práticas de separação de responsabilidades entre camadas de interface, lógica de negócio e persistência de dados.

II. REQUISITOS

A. Requisitos Funcionais

- Cadastro, edição e exclusão de livros, com campos como ISBN, título, autor, gênero, ano de publicação e quantidade de cópias disponíveis;
- Cadastro, edição e exclusão de usuários (patrons), incluindo identificação, nome, contato e histórico de empréstimos;
- Registro de empréstimos e devoluções, com prazo padrão de 14 dias;
- Renovação de empréstimos e atualização automática da disponibilidade de exemplares;

Este trabalho foi desenvolvido como parte da disciplina de Programação Orientada a Objetos.

- Cálculo automático de multas por atraso, no valor de R\$ 2,00 por dia;
- Bloqueio de novas operações para usuários com pendências ou exclusão de itens com empréstimos ativos;
- Geração de relatórios de empréstimos ativos, empréstimos atrasados e histórico por usuário;
- Controle de acesso por perfil de usuário, com dois níveis: Administrador e Bibliotecário;
- Persistência local dos dados em arquivos de texto, preservando o estado do sistema entre execuções.

B. Requisitos de Projeto

- Carregamento do inventário inicial a partir de arquivos no formato CSV (delimitado por `;`), nomeados `books.txt`, `patrons.txt` e `loans.txt`;
- Interface gráfica em Swing organizada em painéis distintos para livros, usuários, empréstimos e relatórios;
- Separação em camadas de modelo, controlador (serviços) e visualização;
- Documentação Javadoc nas principais classes do sistema.

III. DESCRIÇÃO DO PROJETO

O projeto está estruturado em três camadas principais:

- **Modelo** (`src/model/`): classes de domínio, como `Book`, `Patron`, `Loan`, `User`, `Administrator` e `Librarian`;
- **Controlador** (`src/controller/`): classes de serviço responsáveis pelas regras de negócio e pela persistência, como `LibraryManager`, `BookService`, `PatronService`, `LoanService`, `ReportService` e `DataManager`;
- **Visualização** (`src/view/`): classes de interface gráfica, com destaque para `LibraryGUI`, responsáveis por apresentar informações e capturar as ações do usuário.

A classe `Main` é o ponto de entrada da aplicação: inicializa o sistema, exibe a tela de login e instancia o gerenciador central `LibraryManager`. Após a autenticação, o usuário acessa a interface principal e navega entre os painéis de livros, usuários, empréstimos e relatórios.

A Tabela I resume as principais classes do sistema e suas responsabilidades.

TABLE I
PRINCIPAIS CLASSES DO SISTEMA

Classe	Responsabilidade
Book	Representa os livros do acervo
Patron	Representa os usuários/patronos
Loan	Registra empréstimos e devoluções
User	Modelo base para autenticação
Administrator	Perfil de administrador
Librarian	Perfil de bibliotecário
LibraryManager	Lógica de negócio e validações
DataManager	Persistência em arquivos de texto
AuthenticationService	Autenticação de login
BookService	Operações sobre livros
PatronService	Operações sobre usuários
LoanService	Controle de empréstimos
ReportService	Geração de relatórios
LibraryGUI	Interface gráfica principal
Main	Ponto de entrada da aplicação

A. Controle de Acesso

O sistema possui dois perfis de usuário, com permissões distintas, conforme a Tabela II.

TABLE II
PERFIS DE ACESSO E PERMISSÕES

Perfil	Permissões
Administrador	Cadastro, edição e exclusão de livros e usuários; geração de relatórios; controle de permissões
Bibliotecário	Registro de empréstimos e devoluções; consulta ao acervo

IV. INTERFACE DO SISTEMA

A interface gráfica foi construída em Java Swing e organizada em abas, permitindo acesso rápido às funcionalidades de cadastro de livros, cadastro de usuários, controle de empréstimos e geração de relatórios. As Figuras 1 a 9 apresentam os principais mockups implementados.

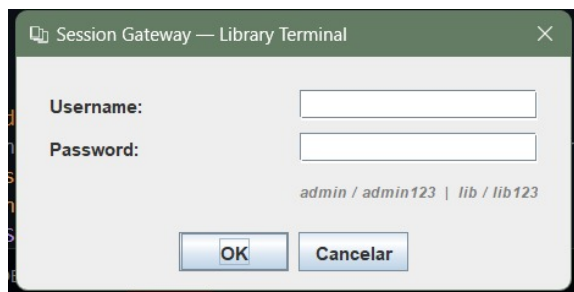


Fig. 1. Tela inicial do sistema

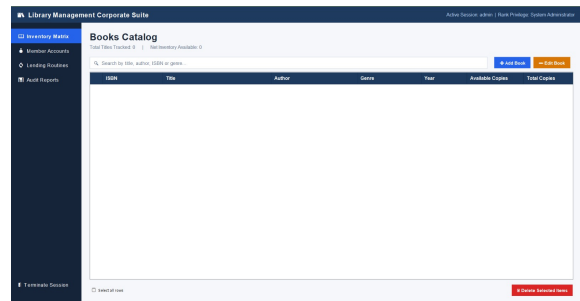


Fig. 2. Tela de listagem de livros

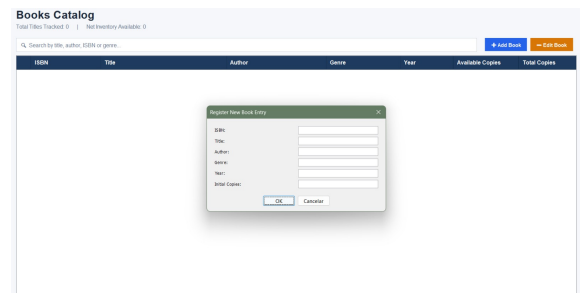


Fig. 3. Formulário de cadastro de livro

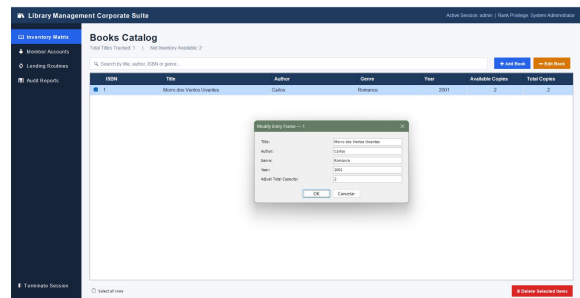


Fig. 4. Formulário de edição de livro

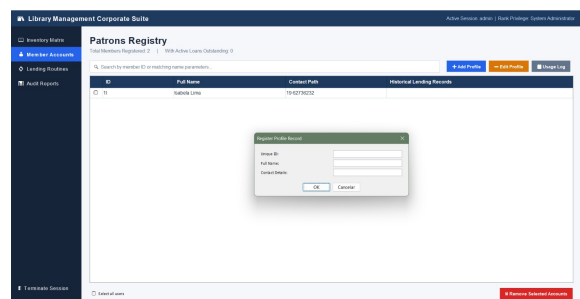


Fig. 5. Confirmação de exclusão de livro

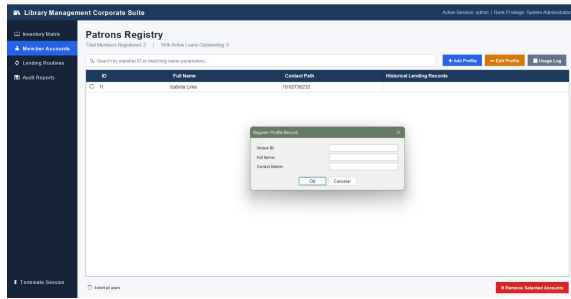


Fig. 6. Formulário de cadastro de usuário (patrono)

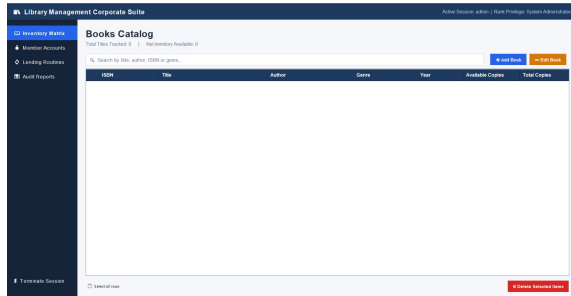


Fig. 7. Tela de histórico de usuários

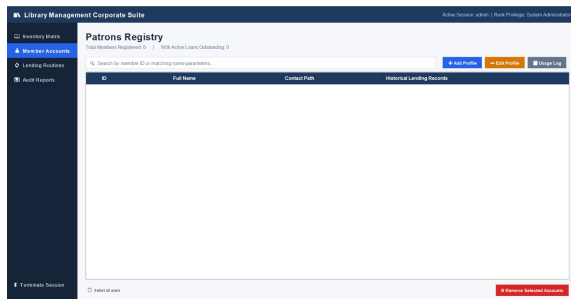


Fig. 8. Tela de controle de empréstimos

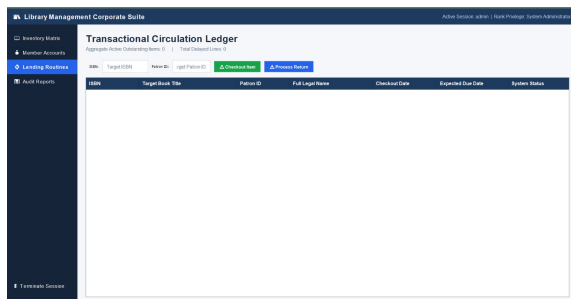


Fig. 9. Tela de geração de relatórios

V. CONCEITOS DE PROGRAMAÇÃO ORIENTADA A OBJETOS APLICADOS

A. Encapsulamento

Os atributos das classes de domínio são protegidos com o modificador `private` e acessados por meio de métodos

`getters` e `setters`, evitando o acesso direto e descontrolado ao estado interno dos objetos.

B. Herança

Os perfis de usuário (Administrator e Librarian) estendem uma classe base (User), reaproveitando atributos e comportamentos comuns de autenticação e especializando apenas o que é específico de cada perfil.

C. Polimorfismo

Por meio da especialização das classes de perfil, operações comuns de autenticação e controle de permissão apresentam comportamentos distintos conforme o tipo concreto do usuário autenticado.

D. Abstração

As classes de domínio (Book, Patron, Loan) modelam apenas os atributos essenciais de cada entidade da biblioteca, abstraindo detalhes de implementação da camada de persistência.

E. Organização em Camadas

A separação entre interface (view), serviços (controller) e persistência (DataManager) contribui para um código mais modular, facilitando a manutenção e a evolução do sistema.

VI. PERSISTÊNCIA DE DADOS

Os dados são mantidos em arquivos de texto no diretório raiz do projeto, em formato CSV delimitado por `;`:

- `books.txt` — acervo de livros;
- `patrons.txt` — cadastro de usuários;
- `loans.txt` — registros de empréstimos e devoluções.

A classe `DataManager` centraliza a leitura e a gravação desses arquivos, sendo acionada sempre que há alterações em livros, usuários ou empréstimos. Essa centralização facilita o controle do formato dos dados, ainda que não ofereça as garantias de integridade de um sistema gerenciador de banco de dados.

VII. COMENTÁRIOS SOBRE O CÓDIGO

A aplicação segue uma organização modular, com classes de serviço responsáveis por fluxos específicos do domínio: `AuthenticationService` cuida da autenticação; `BookService`, `PatronService` e `LoanService` tratam, respectivamente, das operações sobre livros, usuários e empréstimos; `ReportService` é responsável pela geração de relatórios e exportação de logs; e `DataManager` concentra a persistência em arquivo.

Há documentação Javadoc nas principais classes, especialmente em `Main`, `DataManager`, `Book`, `Patron` e `Loan`. A interface gráfica mantém referências diretas ao `LibraryManager`, permitindo que cada painel execute diretamente as operações de domínio. Embora exista separação entre camadas, o acoplamento entre a interface e a lógica de negócio ainda é moderado, sendo um ponto de melhoria para versões futuras.

VIII. PLANO DE TESTES

Foram planejados os seguintes testes manuais:

- Autenticação com as credenciais `admin/admin123` e `lib/lib123`, incluindo casos de credenciais inválidas;
- Verificação do carregamento inicial dos arquivos `books.txt`, `patrons.txt` e `loans.txt`;
- Cadastro, edição e exclusão de livros, com verificação de persistência em arquivo;
- Cadastro e edição de usuários, com verificação de persistência em `patrons.txt`;
- Registro de empréstimo válido, com verificação da redução de cópias disponíveis;
- Devolução de empréstimo, com verificação do aumento de cópias disponíveis;
- Cálculo de multa para empréstimos com devolução em atraso;
- Geração de relatórios de empréstimos ativos, atrasados e histórico por usuário;
- Persistência geral dos dados entre execuções do sistema.

IX. RESULTADOS DOS TESTES

Os testes manuais confirmaram o funcionamento dos principais casos de uso:

- O login funciona corretamente para os perfis de administrador e bibliotecário, incluindo o tratamento de credenciais inválidas;
- Os arquivos de dados são lidos e gravados corretamente nos formatos esperados;
- O cadastro, a edição e a exclusão de livros e usuários estão operacionais;
- Empréstimos e devoluções atualizam corretamente a disponibilidade de exemplares e são persistidos em `loans.txt`;
- O cálculo automático de multas funciona conforme o prazo de 14 dias e o valor de R\$ 2,00 por dia de atraso;
- Os relatórios de empréstimos ativos e atrasados são gerados corretamente, assim como o histórico por usuário.

X. PROCEDIMENTOS DE BUILD

Para compilar e executar o projeto, siga os passos abaixo:

- 1) Instale o Java Development Kit (JDK) 11 ou superior;
- 2) Abra um terminal na pasta raiz do projeto;
- 3) No Windows, execute o script de build:

```
.\run.bat
```

- 4) No Linux/macOS, conceda permissão de execução e rode o script:

```
chmod +x run.sh
./run.sh
```

- 5) Para compilar manualmente:

```
javac -d out src\*.java
java -cp out Main
```

A Tabela III apresenta as credenciais padrão utilizadas para acessar o sistema durante os testes.

TABLE III
CREDENCIAIS PADRÃO DE ACESSO

Perfil	Login	Senha
Administrador	admin	admin123
Bibliotecário	lib	lib123

XI. PROBLEMAS E LIMITAÇÕES

- A persistência é baseada em arquivos de texto simples, sem abstração de banco de dados;
- Ausência de testes automatizados integrados ao repositório;
- Acoplamento parcial entre os componentes de interface gráfica e a lógica de negócio;
- Validação de dados ainda simples em alguns formulários;
- Ausência de um gerenciador de dependências (como Maven ou Gradle) e de um processo de build automatizado.

XII. CONSIDERAÇÕES FINAIS

O JavaLibrary oferece uma base funcional para o gerenciamento de biblioteca, servindo como prova de conceito para a aplicação de conceitos de Programação Orientada a Objetos e de padrões de organização em camadas. Os testes manuais realizados confirmaram o correto funcionamento dos principais fluxos do sistema, incluindo cadastro, empréstimos, devoluções, cálculo de multas e geração de relatórios.

Para versões futuras, recomenda-se: (i) aprofundar a separação entre interface e lógica de negócio, reduzindo o acoplamento atual; (ii) introduzir testes automatizados, cobrindo os principais serviços do sistema; (iii) adotar uma camada de persistência mais robusta, como um banco de dados relacional; e (iv) incorporar um gerenciador de dependências e um pipeline de build automatizado.